

## Part A: Test Plan

### Cinema Distribution System

Document type: Test Plan (Final Project - Part A)  
Group / repository: Enterprise-F-Project / cinema-  
Application stack: Spring Boot 3, Next.js, PostgreSQL, Docker, JWT  
Instructor: Abel Tadesse  
Weight: Course final project (Part A - 10 marks in rubric)

#### Group members

| Name             | Student ID  | Primary testing role                                |
|------------------|-------------|-----------------------------------------------------|
| Zerubabel Mesfin | ATE/0211/14 | Services, unit tests, coverage, GitHub Actions      |
| Hlina Girum      | ATE/3417/14 | Security, JWT, security integration, Jenkins        |
| Mistre Solomon   | ATE/2545/14 | DTOs/controllers, Selenium Page Object system tests |
| Yabsra Mesfin    | ATE/1814/14 | Validation, BVA, EP, controller validation tests    |

This test plan defines how the group will test the Cinema Distribution System. The application is the vehicle for a complete testing effort; the emphasis is on planned, technique-driven, automated testing at unit, integration, and system levels, supported by continuous integration.

## 1. Scope

### 1.1 In scope

- User registration and login with JWT authentication
- Role-based authorization (ADMIN, DISTRIBUTOR, CLIENT)
- Movie management and availability status
- License creation and ownership rules for distributors
- Rental lifecycle (REQUESTED -> ACTIVE -> COMPLETED)
- API / DTO Bean Validation (boundary and equivalence cases)
- Security behaviour for unauthenticated and wrong-role access
- End-to-end UI journeys driven by Selenium (Page Object Model)
- Regression via GitHub Actions and Jenkins pipelines
- Coverage measurement of core business services (JaCoCo)

### 1.2 Out of scope

- Payment gateways and external streaming providers
- Native mobile applications
- Formal load / stress / soak performance campaigns
- Full accessibility (WCAG) certification
- Penetration testing beyond the defined security integration suite

## 2. Approach (levels and techniques)

Testing follows the test pyramid: many fast unit tests, fewer integration/security tests, and a small number of Selenium system tests. Formal techniques (equivalence partitioning, boundary value analysis, decision tables, and state transition testing) drive case design and are reflected in automated suites and the companion Test Design document (Part B).

| Level           | Techniques / tools             | Primary evidence in repository                             |
|-----------------|--------------------------------|------------------------------------------------------------|
| Unit            | Mocks/stubs; EP/decision logic | Auth/Movie/License/Rental *ServiceTest.java                |
| API validation  | BVA + EP on DTOs; MockMvc      | AuthControllerValidationTest + CreateRequestDtoValidati... |
| Integration     | Security + DB (Testcontainers) | SecurityIntegrationTest                                    |
| System          | Selenium + Page Object Model   | CinemaSystemTest and pages/*                               |
| Regression / CI | Automate on every push         | .github/workflows/ci.yml and Jenkinsfile                   |

### 3. Entry criteria

- Application source builds successfully (Maven backend, npm frontend).
- PostgreSQL is available via Docker Compose; Flyway migrations apply cleanly.
- Demo users exist (admin@cinema.com, distributor@cinema.com, client@cinema.com).
- JDK 21, Maven, Node.js/npm, Docker, and Chrome are available where required.
- Test plan and test design baselines are agreed by the group.

### 4. Exit criteria

- CI suite (unit + validation + security) is green on GitHub Actions / Jenkins.
- Selenium system tests pass on a full local stack (frontend :3000, backend :8080, DB, Chrome).
- JaCoCo branch coverage of core business services is at least 90%.
- Defects are logged with severity/priority/status; critical path defects fixed or deferred.
- Test summary report states a clear release/demo recommendation.

### 5. Risk-based prioritisation

Higher risk areas receive denser automated coverage and earlier execution in CI.

| Priority | Area under test                    | Rationale                                             |
|----------|------------------------------------|-------------------------------------------------------|
| P1       | Authentication, JWT, roles         | Security failures have high impact                    |
| P1       | Rental status workflow             | Core stateful business process; user-visible failures |
| P2       | License ownership rules            | Distributor isolation; data integrity                 |
| P2       | Input validation (DTO constraints) | Prevents corrupt data and opaque 500 errors           |
| P3       | UI cosmetics / non-critical layout | Lower business risk; light E2E smoke only             |

### 6. Schedule

| Week   | Focus                                                      |
|--------|------------------------------------------------------------|
| Week 1 | Working application baseline; write test plan (Part A) ... |
| Week 2 | Implement unit, integration, Selenium tests; set up Git... |
| Week 3 | Regression demonstration, defect log, metrics, summary ... |

### 7. Roles and responsibilities

| Member           | Student ID  | Responsibility in the testing effort                |
|------------------|-------------|-----------------------------------------------------|
| Zerubabel Mesfin | ATE/0211/14 | Domain services; unit tests; JaCoCo; GitHub Actions |
| Hlina Gorum      | ATE/3417/14 | Spring Security & JWT; security tests; Jenkinsfile  |
| Mistre Solomon   | ATE/2545/14 | DTOs/controllers; Selenium POM system tests         |
| Yabsra Mesfin    | ATE/1814/14 | Validation, BVA, EP; controller validation tests    |

### 8. Test environment

- Backend: Spring Boot on <http://localhost:8080>

- Frontend: Next.js on http://localhost:3000
- Database: PostgreSQL 16 via Docker (host port 5433 in project compose)
- CI agents: Ubuntu (GitHub Actions) / Jenkins agent with JDK 21, Maven, Docker
- Browsers for system tests: Google Chrome (Selenium Manager resolves ChromeDriver)

## 9. Acceptance / UAT notes

Manual acceptance uses Postman and the web UI with seeded demo accounts (password Password123!). Role-specific journeys (client rental request; distributor movie/license creation; admin oversight) are confirmed before demo.

## 10. Deliverables linked to this plan

- Automated suites in backend/src/test/java (unit, validation, security, selenium)
- CI: .github/workflows/ci.yml and Jenkinsfile
- Coverage: backend/target/site/jacoco/index.html
- Defect log and metrics: docs/defect-log.md, docs/metrics.md
- Test design (Part B) and test summary (Parts H/I) in docs/

*End of Part A - Test Plan.*